

Instantiierung in Java

Technik	Beschreibung	Beispiel	Besonderheiten
Klassische Instantiierung (new-Operator)	Erstellt ein Objekt einer Klasse durch direkten Konstruktoraufruf.	<code>Car car = new Car();</code>	Standardmethode, direkter Speicherverbrauch.
Instantiierung mit Konstruktor-Überladung	Verwendung mehrerer Konstruktoren zur flexiblen Objekterstellung.	<code>Car car = new Car("Red", 2023);</code>	Erlaubt verschiedene Initialisierungsoptionen.
Instantiierung mit Fabrikmuster (Factory Pattern)	Objekt wird über eine Methode erzeugt, nicht direkt mit <code>new</code> .	<code>Car car = CarFactory.createCar();</code>	Kapselt die Erzeugungslogik, fördert lose Kopplung.
Instantiierung mit Singleton-Pattern	Sorgt dafür, dass nur eine Instanz existiert.	<code>Car car = Car.getInstance();</code>	Kontrollierte Instantiierung, nützlich für Ressourcenverwaltung.
Instantiierung mit Builder-Pattern	Objekt wird Schritt für Schritt erstellt.	<code>Car car = Car.builder().color("Red").year(2023).build();</code>	Ermöglicht flexible Objektkonfiguration, oft mit <code>@Builder</code> (Lombok).
Instantiierung mit <code>Supplier<T></code> (Functional Interface)	Erzeugt Objekte über eine Lambda-Funktion.	<code>Supplier<Car> carSupplier = Car::new; Car car = carSupplier.get();</code>	Nützlich für Lazy Instantiation und Streams.
Instantiierung mit <code>ObjectInputStream</code> (Deserialisierung)	Lädt ein Objekt aus einer gespeicherten Datei.	<code>Car car = (Car) objectInputStream.readObject();</code>	Objekte werden aus einer binären Quelle rekonstruiert.
Instantiierung mit <code>Enum</code> (für Singletons empfohlen)	Verhindert mehrfaches Instanzieren durch JVM-Unterstützung.	<code>enum Car { INSTANCE; }</code>	Sicherer als Singleton-Klasse mit <code>private static final</code> .
Instantiierung durch Method References	Erzeugt Objekte ohne expliziten <code>new</code> -Operator.	<code>Supplier<Car> supplier = Car::new; Car car = supplier.get();</code>	Kürzere Syntax für Konstruktoraufrufe in funktionaler Programmierung.